Leitura 2
Basic Artificial Neural Network and Architectures

Paulo Vieira

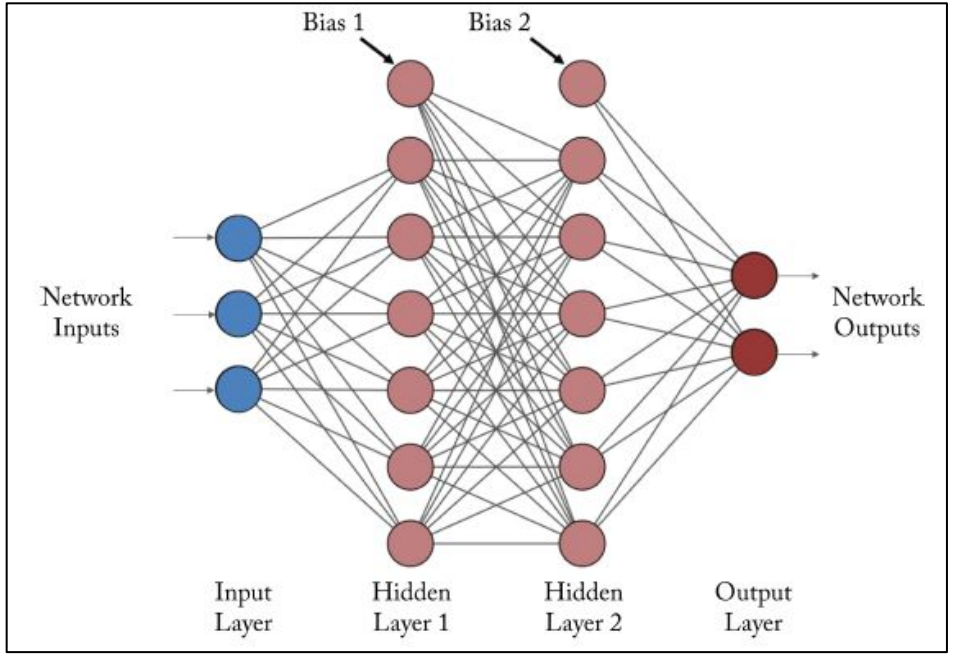
Agenda

Multi-layer Neural Network Architecture
Neurons
Activation functions
MINIMIZING THE LOSS FUNCTION
GRADIENT DESCENT ALGORITHM

Multi-layer Neural Network Architecture

the software architecture of machine learning uses: activation function and connected pathways that can be compared to the biological neurons and synapses

The neural network architecture consists of several basic components, such as the number of inputs, outputs, hidden layers, and the type of activation functions. A simple multi-layer neural network has a topology similar



Multi-layer neural network architecture with two hidden layers.

An artificial neural network can learn (alternatively, find optimum model weights or biases) through training, which is an important phenomenon that mimics the process of learning in our human brain

The learning process of an artificial neural network is actually finding out weights and biases that will generate a minimum error

Multi-layer Neural Network Architecture

Several methods of learning schemes are widely used
in machine learning based on data types or observation methods such as:

- Supervised
- unsupervised, and
- reinforcement learning.

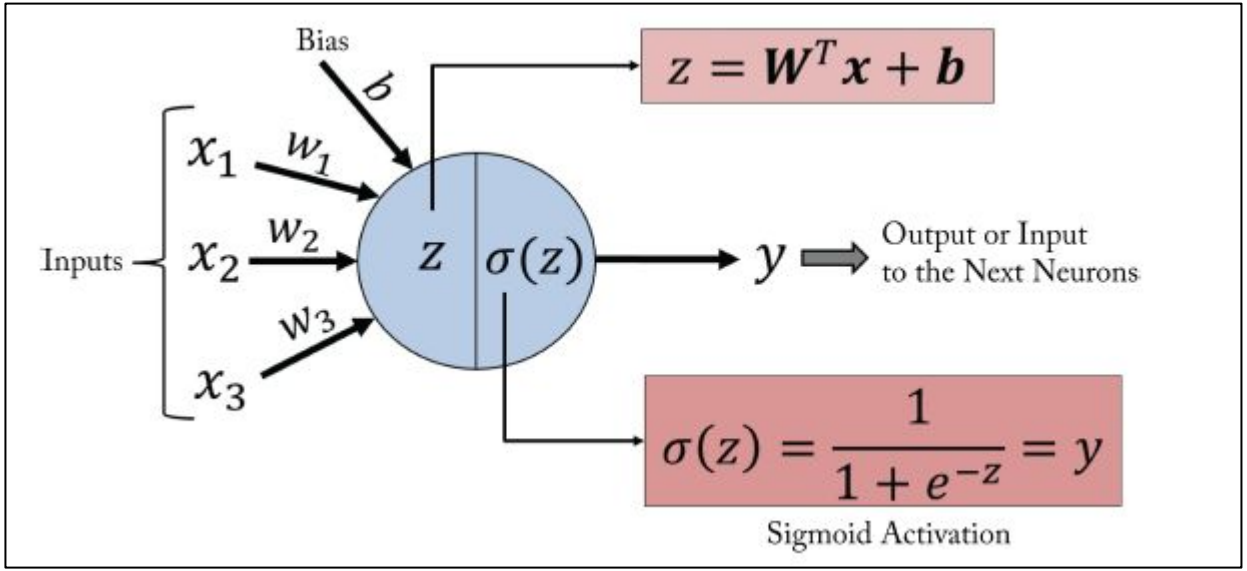
When the network can compare its predictions with the correct results to make adjustments, then it is called supervised learning.

Unsupervised learning is the type of learning when the correct results of the example dataset are unknown. And,

the reinforcement learning are used when learning through observations from a complex environment is required.

Neurons

Many internal parameters can influence the output found from an artificial neural network. But generally, outputs are directly related to the number of neurons and their activation functions.



Computational steps inside one neuron of the hidden layer. Here, $\sigma(z)$ is the sigmoid activation, x_1, x_2, x_3 are inputs to the neuron, w_1, w_2, w_3 are the weights, and b is the bias to the input.

The neurons are the nodes, typically shown by circles that are arranged inside the layers.

Inside a layer, these neurons take input weights from all connections and add a bias, initializes these parameters, and then calculates the output using another function known as an activation function.

The mathematical operations happening inside each layer amplify dramatically as the neurons and connections are increased. F

Neurons

This operation can be divided into two steps of computations.

First, Z is calculated using the weights (W), bias (b), and inputs (x).
Second, the activation function is calculated using Z.

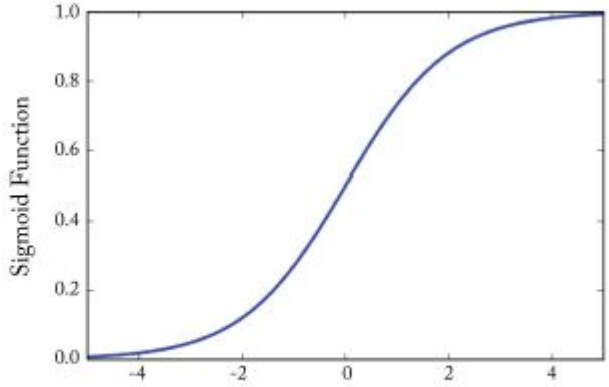
We could simply say that output of a neuron is a function of a function.

In figure above a sigmoid activation function is shown as an example. The sigmoid activation function has the ability to map any predicted value between 0 and 1.

This output can be used to estimate the probability, where a threshold or tipping point (e.g., 0.5) is used for binary classification problems. For example, if output 0:5, class D 1 and if output < 0:5, class D 0.

There are many other types of activation functions available for an artificial neural network.

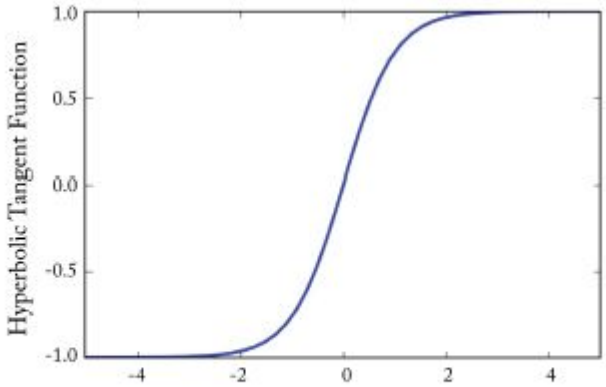
Activation functions: Sigmoid function, Hyperbolic tangent function



Plot of Sigmoid activation function in between −5 to +5.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)).$$

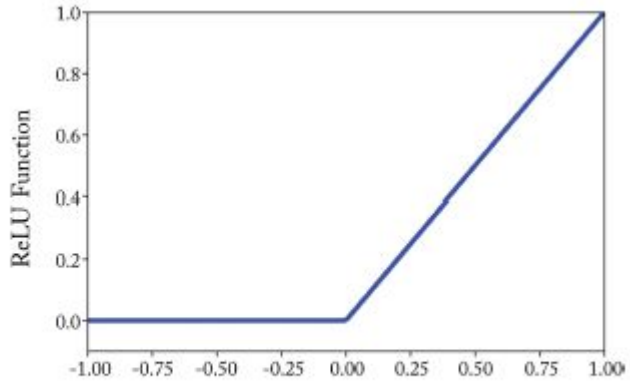


Plot of hyperbolic tangent activation function in between −5 to +5.

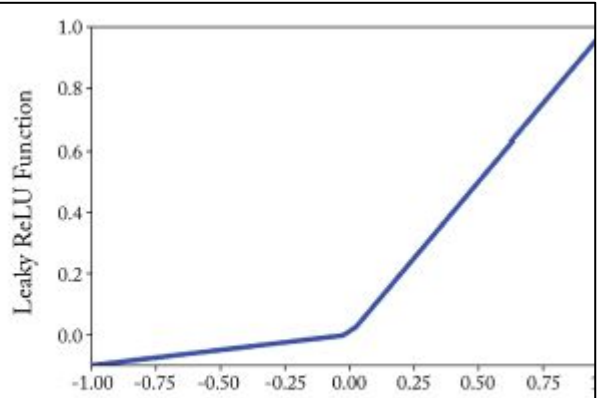
$$\sigma(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

$$\sigma'(z) = \left(1-\sigma(z)^2\right).$$

Activation functions: ReLU function, Leaky ReLU function



Plot of ReLU activation function in between −1 to +1.



Plot of Leaky ReLU activation function in between −1 to +1

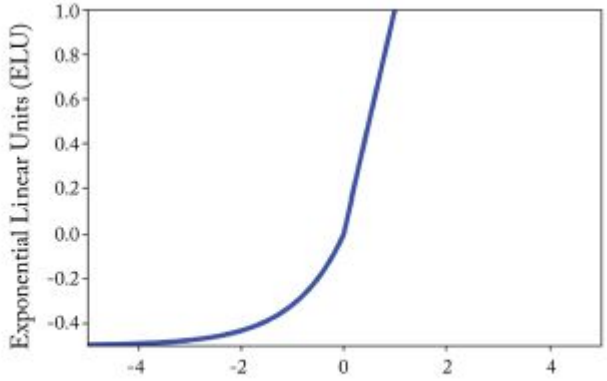
$$\sigma(z) = \begin{cases} 0 & \text{for } z \leq 0 \\ z & \text{for } z > 0 \end{cases}$$

$$\sigma'(z) = \begin{cases} 0 & \text{for } z \leq 0 \\ 1 & \text{for } z > 0. \end{cases}$$

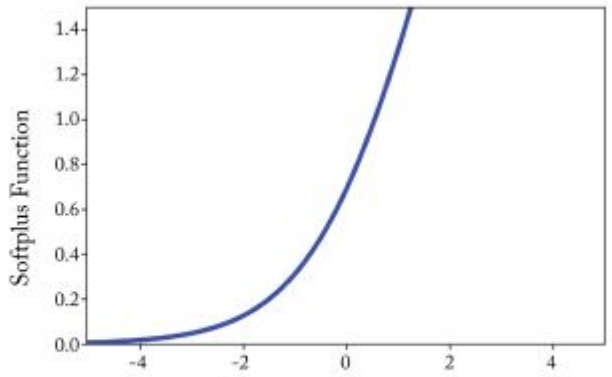
$$\sigma(z) = \begin{cases} az & \text{for } z \leq 0 \\ z & \text{for } z > 0 \end{cases}$$

$$\sigma'(z) = \begin{cases} a & \text{for } z \leq 0 \\ 1 & \text{for } z > 0. \end{cases}$$

Activation functions: ELU function, Softplus function



Plot of ELU activation function in between −5 to +5.



Plot of Softplus activation function in between −5 to +5.

$$\sigma(z) = \begin{cases} a\,(e^z - 1) & \text{for } z \le 0 \\ z & \text{for } z > 0 \end{cases}$$

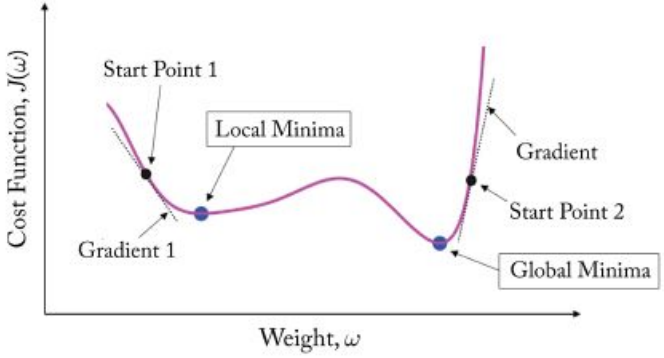
$$\sigma'(z) = \begin{cases} f(z) + a & \text{for } z \le 0 \\ 1 & \text{for } z > 0. \end{cases}$$

$$\sigma(z) = \ln(1 + e^x)$$

$$\sigma'(z) = \frac{1}{1 + e^{-x}}.$$

MINIMIZING THE LOSS FUNCTION

$$J = -\frac{1}{M}\sum_{m=1}^{M}\left[y_m \times \log\left(h_\theta\left(x_m\right)\right) + \left(1 - y_m\right) \times \log\left(1 - h_\theta\left(x_m\right)\right)\right],$$



If the initial starting point is Start Point 1, the gradient descent algorithm will converge to a local minima. On the other hand, if the Start Point 2 is used, the algorithm will converge to the global minima.

where M = number of training examples, y_m = target label for training example m, x_m = input for training example m, h_θ = model with neural network weights θ. For categorical targets if $\hat{y}_1 \ldots \hat{y}_m$ are the probabilities of the "m" classes, and the rth class is the ground-truth class, then

$$J = -\log\left(\hat{y}_r\right).$$

The goal of any neural network is to minimize this equation. This minimization process is done by updating weights using an optimization function or optimizer.
An important iterative algorithm known as backpropagation is used to apply this optimization, and in this process, the loss function dictates how well the network is learning

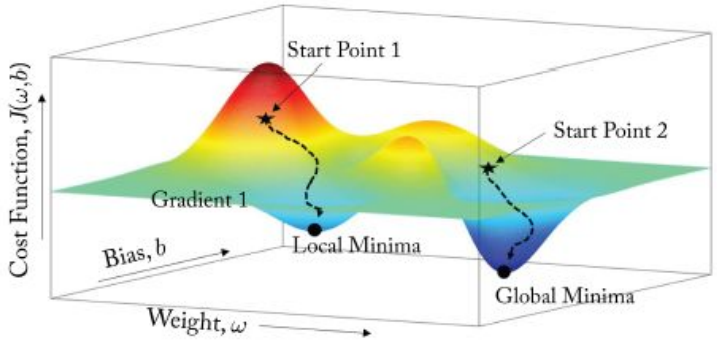
Many different optimizers have been used in artificial neural networks such as: gradient descent, gradient descent with momentum, Root Mean Square Propagation (RMSProp), Adam optimizers, and so on.

The selection of a suitable optimizer depends on the computational scenario, and they typically differ in terms of their simulation speed and convergence ability

GRADIENT DESCENT ALGORITHM

The gradient descent is an iterative first-order optimization algorithm that helps the neural network to minimize the cost function, J .

In the case of linear regression, we can use derivatives to find a minimum of convex functions. But in the artificial neural network, we always deal with a non-convex function, and therefore gradient descent algorithm can end up finding the local minima instead of global minim

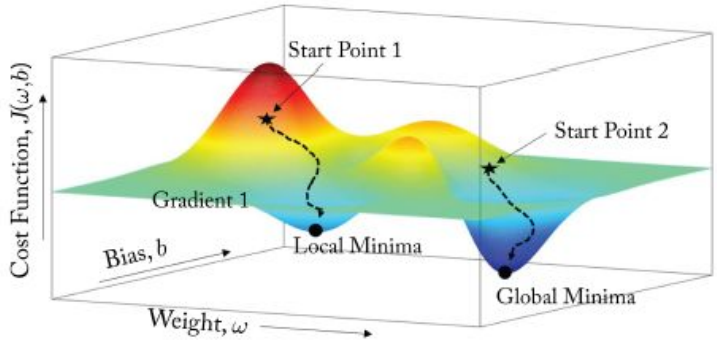


If the Cost function, J is a function of two parameters (weight and bias), the gradient descent algorithm will search for a minima on the surface. In this example plot, the Start Point 1 will converge to a local minima, and the Start Point 2 will converge to a global minima.

GRADIENT DESCENT ALGORITHM

The gradient descent algorithm is simultaneously performed for all values of weights and biases. If we have n neurons in a hidden layer, all the weights will update through Equation . Here α is the learning rate, which tells the algorithm how big or small the step size would be while moving in the direction of downhill steep. The negative gradients push the algorithm to move toward the downhill directions.

$$\omega_1 = \omega_1 - \alpha \frac{\partial}{\partial \omega_1} J(\omega, b)$$
$$\omega_2 = \omega_2 - \alpha \frac{\partial}{\partial \omega_2} J(\omega, b)$$
$$\omega_3 = \omega_3 - \alpha \frac{\partial}{\partial \omega_3} J(\omega, b)$$
$$\vdots$$
$$\omega_n = \omega_n - \alpha \frac{\partial}{\partial \omega_n} J(\omega, b).$$



C If the Cost function, J is a function of two parameters (weight and bias), the gradient descent algorithm will search for a minima on the surface. In this example plot, the Start Point 1 will converge to a local minima, and the Start Point 2 will converge to a global minima.

The bias parameters for a hidden layer will update using Equation (3.16):

$$b = b - \alpha \frac{\partial}{\partial b} J(\omega, b).$$

Example of an ANN model

```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# 1. Carregar o dataset MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 2. Normalizar os valores dos píxeis para o intervalo [0, 1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# 3. Transformar as labels em one-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 4. Construir o modelo
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),   # 28x28 -> 784
    tf.keras.layers.Dense(512, activation="relu"),
    tf.keras.layers.Dense(256, activation="relu"),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

```python
# 5. Compilar o modelo
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# 6. Treinar o modelo
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

# 7. Avaliar o modelo
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

print("Test loss:", test_loss)
print("Test accuracy:", test_accuracy)

# 8. Guardar o modelo treinado
model.save("mnist_model_tf2.keras")
```

FIM